

1. Programming

1. What are the 4 pillars of OOP?

- **Encapsulation:** Protects data inside a class.
 - **Abstraction:** Hides implementation details.
 - **Inheritance:** Reuses code from parent classes.
 - **Polymorphism:** Same method behaves differently for different objects.
-

2. Design a WMS class structure.

Warehouse → **Inventory** → **Product** → **Shipment** → **Vehicle** → **Driver** where **Inventory** manages stock and **Shipment** tracks deliveries.

3. How do you calculate ETA from GPS?

Calculate the distance using GPS coordinates (Haversine Formula), then **ETA = Distance ÷ Average Speed**.

4. Multithreading vs Multiprocessing?

- **Multithreading:** Best for I/O tasks like GPS tracking and API calls.
 - **Multiprocessing:** Best for CPU-intensive tasks like route optimization.
-

5. How do you deduplicate shipment records?

Use a unique Shipment ID with a dictionary or set to remove duplicate entries.

6. What is a memory leak?

Memory that is allocated but never released, causing increasing memory usage over time.

7. How do you handle failed GPS API calls?

Use **try-except**, log the error, retry with exponential backoff, and notify if retries fail.

8. Abstract class vs Interface?

- **Abstract Class:** Can have implemented methods and state.
 - **Interface:** Defines only behavior (contracts); used for multiple implementations.
-

9. Detect a vehicle stopped for 30 minutes.

Compare the current GPS location with the previous one; if unchanged for 30 minutes, raise an alert.

10. What is a Design Pattern?

A reusable solution to common design problems (e.g., **Observer** for real-time tracking updates).

11. Prevent concurrent inventory updates.

Use **database transactions**, **locks**, or **optimistic concurrency control**.

12. Why is sorting important?

Sorting by **priority** or **ETA** helps process urgent shipments first.

2. SQL & Database

1. Find delayed shipments.

Use SQL with a date difference condition, e.g., `DATEDIFF(CURDATE(), expected_date) > 2`.

2. Join Orders, Inventory, and Shipments.

Use **INNER JOIN** on their foreign keys.

3. Optimize slow SQL queries.

Create indexes, avoid unnecessary joins, and analyze the execution plan.

4. Normalization vs Denormalization.

- **Normalization:** Reduces redundancy.
 - **Denormalization:** Improves read performance.
-

5. Inventory schema.

Warehouse → Product → Inventory → Shipment tables connected with foreign keys.

6. SQL vs NoSQL.

- **SQL:** Structured data with ACID transactions.
 - **NoSQL:** Flexible schema and better horizontal scalability.
-

7. What is an Index?

An index speeds up data retrieval but slows inserts and updates.

8. What are ACID properties?

Atomicity, Consistency, Isolation, Durability ensure reliable database transactions.

9. INNER vs LEFT vs RIGHT JOIN.

- **INNER:** Matching rows only.
 - **LEFT:** All left rows + matches.
 - **RIGHT:** All right rows + matches.
-

10. Zero-downtime migration.

Use rolling deployments, backward-compatible schema changes, and versioned APIs.

11. What is a deadlock?

Two transactions wait on each other indefinitely; resolve by rollback or consistent locking order.

12. Top 5 warehouses by shipment volume.

Use `GROUP BY`, `COUNT()`, `ORDER BY DESC`, and `LIMIT 5`.

13. Primary Key vs Foreign Key vs Composite Key.

- **Primary:** Unique identifier.
 - **Foreign:** References another table.
 - **Composite:** Combination of multiple columns as a key.
-

3. Cloud Computing

1. Cloud services used?

AWS EC2 for servers, S3 for storage, and Docker for deployment.

2. Handle GPS data from 10,000 vehicles.

Use a Load Balancer, Message Queue (Kafka/SQS), scalable servers, and databases.

3. Horizontal vs Vertical Scaling.

- **Horizontal:** Add more servers.
 - **Vertical:** Increase server resources.
-

4. Ensure High Availability.

Deploy multiple server instances behind a Load Balancer.

5. What is a Load Balancer?

Distributes traffic across multiple servers to improve availability and performance.

6. IaaS vs PaaS vs SaaS.

- **IaaS:** Virtual machines.
 - **PaaS:** Managed platform.
 - **SaaS:** Ready-to-use software.
-

7. Use AWS Lambda?

Trigger alerts or process GPS events without managing servers.

8. What is a Message Queue?

Stores messages temporarily for asynchronous processing (e.g., Kafka, Amazon SQS).

9. Disaster Recovery?

Use automated backups, replication, and failover systems.

10. Auto Scaling?

Automatically adds or removes servers based on traffic.

11. Docker vs Virtual Machine.

Docker shares the host OS, while a VM includes a full operating system.

12. Monitor performance.

Use CloudWatch, Prometheus, or Grafana with alerts.

4. Networking

1. TCP vs UDP.

- **TCP:** Reliable and ordered.
 - **UDP:** Faster with no delivery guarantee.
-

2. REST API for shipment status.

Use GET /shipments/{id} to return the current shipment status.

3. GPS device loses connection.

Check device power, network connectivity, server logs, and API responses.

4. What is DNS?

Converts domain names into IP addresses.

5. LAN vs WAN.

- **LAN:** Local network.
 - **WAN:** Connects multiple distant locations.
-

6. Firewall?

Blocks unauthorized network access.

7. VPN?

Provides secure remote access to internal systems.

8. HTTP Request/Response Cycle.

Client sends a request, server processes it, and returns a response.

9. Packet Loss?

Packets fail to reach the destination; diagnose using ping or traceroute.

10. HTTP vs HTTPS.

HTTPS encrypts data using SSL/TLS.

11. Common HTTP Status Codes.

- 200: Success
 - 401: Unauthorized
 - 404: Not Found
 - 500: Server Error
-

12. Rate Limiting?

Limits API requests to prevent abuse and server overload.

5. IoT & GPS

1. GPS data flow.

GPS Device → Internet → Cloud Server → Database → Dashboard.

2. What is MQTT?

A lightweight messaging protocol for IoT devices.

3. GPS data gaps.

Store the last known location and update when connectivity returns.

4. GPS vs RFID.

GPS tracks outdoors; RFID tracks items indoors.

5. MQTT QoS.

Defines the reliability level of message delivery.

6. Detect fake GPS data.

Validate location, speed, and movement against expected values.

7. Real-time vs Batch tracking.

Real-time updates instantly; batch sends updates at intervals.

8. Estimate ETA.

Use GPS location, speed, distance, and traffic data.

9. Battery challenges.

Reduce transmission frequency and use low-power communication.

10. Store high-frequency GPS data.

Use message queues, stream processing, and scalable databases.

11. What is Geofencing?

Creates a virtual boundary to trigger alerts when vehicles enter or leave an area.

12. Remote firmware updates.

Use secure OTA (Over-the-Air) updates.

6. WMS / TMS / ERP

1. WMS vs TMS.

WMS manages warehouses; TMS manages transportation and deliveries.

2. Integrate WMS with ERP.

Use APIs or middleware to synchronize inventory and orders.

3. Real-time dashboard.

Display shipment status, GPS location, ETA, and alerts.

4. Ensure data security.

Use authentication, encryption, role-based access, and HTTPS.

5. Cross-docking.

Transfers goods directly from inbound to outbound trucks with minimal storage.

6. FIFO vs LIFO.

FIFO uses the oldest stock first; LIFO uses the newest stock first.

7. What is a Pick List?

A list of items warehouse workers collect for an order.

8. Inventory discrepancy.

Perform a physical audit and update the system accordingly.

9. Barcode/RFID role.

Automatically identifies products and updates inventory records.

10. Support multiple carriers.

Create a common API interface for different logistics providers.

11. What is EDI?

Electronic Data Interchange enables standardized business document exchange.

12. ERP and WMS stock mismatch.

Run reconciliation jobs and define a single source of truth.

7. Troubleshooting

1. Wrong shipment location.

Check GPS device, API, database, and mapping service.

2. Dashboard is slow.

Analyze SQL queries, API latency, and server resources.

3. Nightly inventory job failed.

Check logs, scheduler, and notifications; enable automatic retries.

4. System is down.

Check monitoring dashboards to identify whether the issue is with the app, database, or network.

5. Carrier API stopped working.

Verify API status, authentication, and network connectivity.

6. Memory leak.

Use profiling tools to identify objects that aren't being released.

7. Warehouse not syncing.

Check network, APIs, message queues, and database replication.

8. Code bug vs Infrastructure issue.

Review application logs, monitoring metrics, and server health.

9. Logging and Monitoring.

Use centralized logging, metrics dashboards, and automated alerts.

10. Critical production bug.

Stabilize the system first, deploy a hotfix, and monitor the results.

11. Wrong GPS timestamp.

Validate timestamps on the server and synchronize using NTP.

8. Behavioral Questions

1. Multi-component project?

Discuss your **ArogyaAI** project with FastAPI, Docker, APIs, databases, and deployment.

2. Solved a difficult problem?

Explain the issue, your approach, the solution, and the outcome.

3. Prioritize work?

Address production issues first, then handle planned feature development.

4. Learned a new technology?

Mention quickly learning Docker, FastAPI, or LangGraph for a project.

5. Technical disagreement?

Listen to teammates, compare solutions, and choose the best option using evidence.

6. Working with non-technical teams?

Use simple language and avoid technical jargon.

7. Automation example?

Describe automating data processing or deployment in your project.

8. Project mistake?

Admit the mistake, explain the fix, and highlight what you learned.

9. Unclear requirements?

Ask clarifying questions, build a prototype, and confirm expectations.

10. Explain technical issues simply?

Use analogies and focus on business impact instead of technical details.

11. Manage multiple tasks?

Prioritize by urgency and use a task-tracking tool.

12. Where will you contribute most?

Backend development, API integration, database optimization, and logistics automation using Python and cloud technologies.